RELAYSPEC: REUSING FROZEN DRAFTERS ACROSS LANGUAGE MODEL SIZES

Anonymous authors
Paper under double-blind review

ABSTRACT

A speculative drafter trained on one language model expects that model's features. Reusing it with a new target therefore requires a compatible input. RelaySpec learns a linear map from the new target's hidden states to the context consumed by the frozen drafter. The source model supplies fitting features, then leaves the deployed conditioning path. We fit the map on 4,096 problem-and-solution records and retarget Qwen3-4B DFlash and EAGLE-3 drafters to Qwen3-8B and Qwen3-14B. On MATH-500, measured end-to-end throughput is 2.35–5.11 times plain autoregressive decoding. The relay retains 89.1–90.3% of native target-specific drafter throughput in paired comparisons. Math accuracy differs from AR by -0.2 to $+0.8$ percentage points. Fitting-budget, normalization and block-size studies relate accepted progress to throughput. Separate EAGLE-3 measurements show 7.63–7.80 GiB lower peak allocated memory than source reuse. These results establish the measured cost and performance of fitting a new interface while retaining an already trained drafter.

1 INTRODUCTION

A deployed language model may change while its available speculative drafter remains fixed. For example, a developer may have a DFlash drafter trained for Qwen3-4B but want to serve Qwen3-8B. The new target can check proposed tokens, but its hidden states do not directly match the input learned by the 4B drafter. Training a new target-specific drafter solves this compatibility problem by learning another drafting model. RelaySpec instead learns the connection to the existing one.

Speculative decoding accelerates autoregressive (AR) generation by checking several proposed tokens in one target pass (Leviathan et al., 2023). DFlash and EAGLE-3 improve proposals using features computed inside a language model (Chen et al., 2026; Li et al., 2025). We call the model that provided these features during drafter training the *source*. The *target* is the model whose token choices govern deployment. Running the source alongside a new target supplies compatible features, but also retains its transformer computation and memory.

RelaySpec fits a map from features already computed by the new target to the context consumed by the frozen drafter. The source supplies training features, then leaves the deployed conditioning path. Only the map is trained. The target, drafter, and inherited token embedding and vocabulary projection remain fixed.

The evaluation asks three questions. How much faster is retargeted speculative decoding than plain AR? How much of a target-specific drafter's throughput does it retain? How much new fitting data and work are required? Figure 1 addresses the first question. The native-drafter and training-budget comparisons address the other two. Source reuse is a secondary control that isolates the work removed by the map.

Our contributions are: (1) an interface-specific procedure for retargeting two frozen drafter families, (2) paired AR, native-drafter and source-reuse comparisons with absolute throughput and task scores, and (3) experiments relating fitting budget, normalization and draft length to throughput, accepted progress and memory. The demonstrated setting is low-cost adaptation of an available drafter, with a new map for each target.

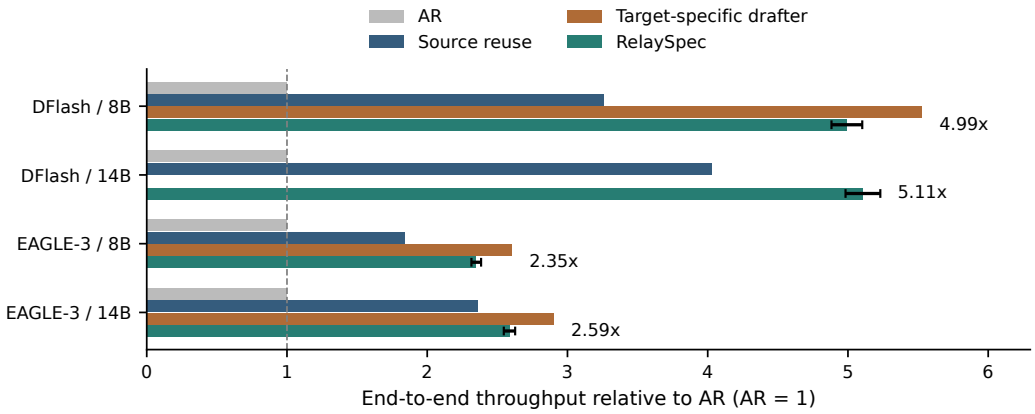

Figure 1: MATH-500 throughput relative to plain AR, measured within each paired run. RelaySpec reuses the 4B drafter. The native control uses a drafter released for the evaluated target. Whiskers show RelaySpec's 95% paired intervals. Native controls show the three evaluated releases. Each runtime uses one request per GPU.

Table 1: Closest approaches and the operation each learns. This is a method comparison. Our measured runtime controls are reported separately in Table 3.

Approach	Learned operation	Relation to retargeting
DFlash / EAGLE-3	Train a drafter using features of its target	Native target-specific performance reference
PARD (An et al., 2026)	Adapt a model for parallel drafting	Reuse a target-independent drafter across targets
SD^2 (Berdoz et al., 2026)	Inject target-derived steering into a pretrained drafter	Align an existing drafter using target information
RepSpec (Huo et al., 2026)	Expand linear structure during training, merge for inference	Improve drafter training with fixed deployment structure
Draft-OPD (Lei et al., 2026)	Update a drafter on states exposed by its proposals	Improve acceptance through new drafter training
RelaySpec	Map new-target features into a frozen drafter's input	Replace source-transformer conditioning with a fitted map

2 Related work

Feature-conditioned drafting. EAGLE predicts future features while conditioning on shifted tokens (Li et al., 2024a). EAGLE-3 combines multiple feature layers with simulated drafting steps (Li et al., 2025). DFlash instead predicts a block in parallel (Chen et al., 2026), while Medusa uses several prediction heads (Cai et al., 2024). GRIFFIN addresses token alignment between training and decoding (Hu et al., 2025). RepSpec expands linear structure during training and merges it for inference (Huo et al., 2026). VSD and Draft-OPD train toward accepted proposals (Zou et al., 2026; Lei et al., 2026). These methods improve drafting itself. RelaySpec learns the input to a drafter whose weights remain fixed.

Reusing trained models. PARD adapts a model for parallel drafting and supports reuse across targets (An et al., 2026). SD^2 steers pretrained drafters with target information, including frozen variants (Berdoz et al., 2026). OmniDraft combines cross-vocabulary and online adaptation (Ramakrishnan et al., 2025). Verification across different vocabularies is also studied directly (Timor et al., 2025a). RelaySpec addresses a shared-vocabulary setting: replacing the original source transformer's conditioning features with mapped target features.

Small trainable modules and low-rank weight updates are established ways to adapt frozen models (Houlsby et al., 2019; Hu et al., 2022). Learning connections between frozen networks is studied in model stitching (Bansal et al., 2021). Our map supplies a frozen drafter's input rather than modifying its internal weights. We measure decoding behavior and runtime, since a useful connection alone does not establish that two networks encode the same information (Smith et al., 2025).

Proposal structure and execution. EAGLE-2 and OPT-Tree adapt draft trees to improve accepted length (Li et al., 2024b; Wang et al., 2025). SpecInfer organizes tree-based proposal verification (Miao et al., 2024). Distributed speculative inference and speculative speculative decoding overlap work across model instances or execution stages (Timor et al., 2025b; Kumar et al., 2026). These choices affect cycle cost and can be studied separately from the feature map. Our comparisons hold the released decoder and proposal structure fixed within each drafter family.

Drafting within the target. Draft & Verify skips target layers during drafting (Zhang et al., 2024). LayerSkip trains early exits and shares computation with verification (Elhoushi et al., 2024). Lookahead decoding generates and checks candidates without an auxiliary drafter (Fu et al., 2024). RelaySpec serves a different deployment choice: retaining an available external drafter when changing the target.

3 PRELIMINARIES

3.1 DRAFTING AND TARGET VERIFICATION

An autoregressive model generates one token at a time. Blockwise and speculative methods propose several tokens, then check them in a target pass (Stern et al., 2018; Xia et al., 2023; Leviathan et al., 2023). In greedy decoding, the verifier commits the longest matching prefix and a target-selected next token. RelaySpec preserves this decoder and changes the features supplied to its drafter.

3.2 THROUGHPUT AND ACCEPTED PROGRESS

Our primary metric is end-to-end output throughput: total generated tokens divided by total request time, including prompt processing. Let T_A, T_R and T_N denote this throughput for plain AR, RelaySpec and the native target-specific drafter. We report

$$\text{AR speedup} = T_R/T_A, \qquad \text{native throughput retained} = 100\,T_R/T_N\ \%. \tag{1}$$

A retention value of 90% means the reused drafter produces 90% as many tokens per second as the target's own drafter. It is not the fraction of speedup above AR. We also report the ratio of summed AR request time to summed relay request time. This differs from a throughput ratio when output lengths differ.

Acceptance explains how speed is obtained. We report the pooled number of tokens advanced per proposal-verification cycle, including the implementation's target-supplied progress. This is distinct from the fraction of proposed draft tokens accepted. Appendix D gives the position-wise curves and precise counting convention.

Let c_R be the average relay cycle time and a_R its average progress. Its decode time per output token is approximately c_R/a_R. If c_A is AR time per token, then the decode-only approximation is

$$\text{AR-relative decode speed} \approx \frac{a_R c_A}{c_R}. \tag{2}$$

Removing source conditioning reduces cycle cost. Imperfect feature matching may reduce progress. The map is useful when the saved computation outweighs the extra cycles. We measure end-to-end speed directly and use this approximation to interpret the component measurements.

4 METHOD

4.1 FIT THE INPUT CONSUMED BY THE FROZEN DRAFTER

We run the source and new target on the same text and collect five hidden states at each token position. A hidden state is the internal feature vector after a transformer block. Joining the selected target

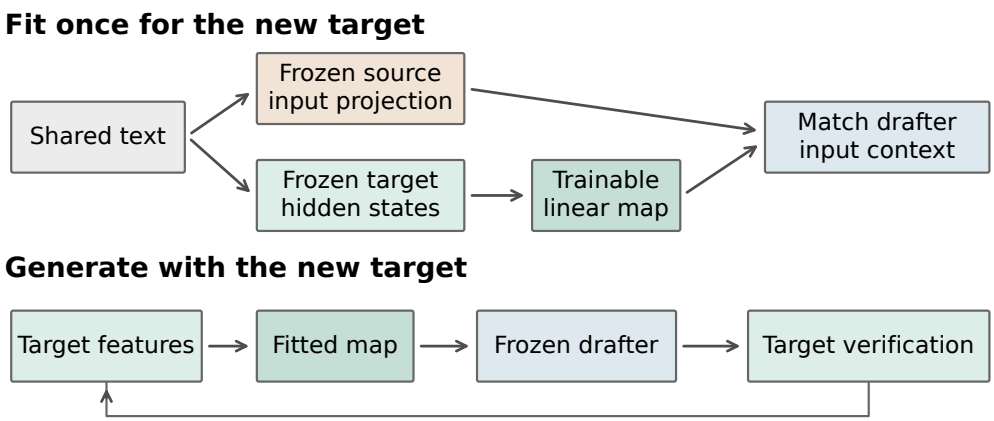

Figure 2: Fit once, then reuse the frozen drafter. The source and target process identical text during fitting. Only the map is updated. During generation, target features from committed positions feed the next draft.

states gives h_t at position t. The corresponding source vector is s_t. The drafter's frozen projection F converts source features to its input width. We learn a bias-free matrix W that supplies this input from h_t. Figure 2 separates fitting from generation.

The correct input depends on the released drafter. DFlash consumes a normalized projection. Let N_{in} denote fixed root-mean-square normalization of joined target features (Zhang & Sennrich, 2019), which rescales a vector by the square root of its mean squared coordinate. Let N_{out} be the released drafter's frozen output normalization. EAGLE-3 consumes the raw projection. The teacher context c_t and mapped context $\widehat{c}_t$ are

$$\begin{aligned} \text{DFlash:} \quad & c_t = N_{\text{out}}(Fs_t), \quad \widehat{c}_t = N_{\text{out}}(WN_{\text{in}}(h_t)), \\ \text{EAGLE-3:} \quad & c_t = Fs_t, \qquad\quad \widehat{c}_t = Wh_t. \end{aligned} \tag{3}$$

Preserving feature magnitude matters for the raw EAGLE-3 input. DFlash's output normalization is applied to both teacher and prediction. The normalization ablation tests these choices directly.

We fit W by minimizing the squared context error relative to the teacher's squared magnitude. For a record with n token positions,

$$\mathcal{L}(W) = \frac{1}{n}\sum_{t=1}^{n} \frac{\|\widehat{c}_t - c_t\|_2^2}{\max(\|c_t\|_2^2, \epsilon)}. \tag{4}$$

Here $\|v\|_2^2$ sums the squared coordinates of v, and ϵ is the smallest positive normal FP32 value. Dividing by teacher magnitude prevents larger context vectors from dominating the objective. We average positions within each record and record losses across workers. DFlash's normalization makes this objective different from raw linear least squares. Both selected maps are fitted with gradient updates.

Fitting uses 4,096 distinct math problem-and-solution records rendered as conversations, with the resulting text truncated to 192 tokens. The supervision is a source context vector. Solution text remains part of the input. The fitted map has 52.43M weights for an 8B target and 65.54M for a 14B target. Appendix A specifies the layers and dimensions.

4.2 Draft with the new target's features

The target first processes the prompt and retains its attention cache and selected hidden states. The map supplies drafter context. The frozen drafter proposes tokens, and the target checks them in parallel with a causal mask. The decoder commits the longest matching prefix and the target-selected

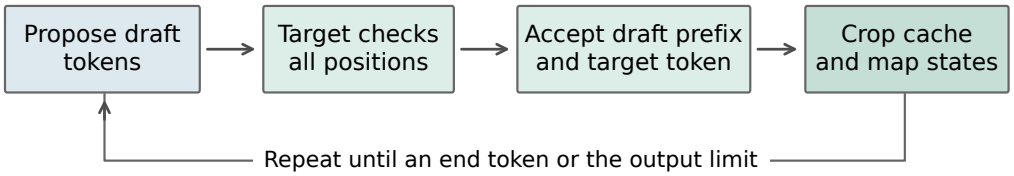

Figure 3: One generation cycle. RelaySpec changes the drafter's context. The target retains control over committed tokens and the shared decoder retains responsibility for positions, stopping and cache updates.

Table 2: Evaluation protocol. Main fitting choices remain fixed across workloads. Full identifiers and runtime versions are in Appendix A.

Component	Setting
Primary baseline	Plain greedy AR with the same target, prompt and runtime
Other controls	Native target-specific drafter and optimized source reuse
Generation	BF16, SDPA attention, thinking disabled, maximum 2,048 new tokens
Relay fitting	4,096 records, 192-token cap, 1,024 AdamW updates (Loshchilov & Hutter, 2019), four records/update
Optimizer	Learning rate 6×10^{-4}, zero weight decay, gradient clipping 1.0
Timing	Synchronized request timer, warmup excluded, rotated method order
Uncertainty	10,000 paired resamples, whole conversations for two-turn chat

next token, then crops the cache to the committed prefix. Mapped target states supply the next cycle, as shown in Figure 3.

The standard greedy equivalence argument requires block verification to return the same token choices as single-token evaluation of each identical prefix (Leviathan et al., 2023). Finite-precision implementations can change those choices. We therefore measure task accuracy and token agreement separately from throughput. The map itself never supplies the scores used for target verification.

5 Experiments

5.1 Experimental setup

Models and runtime. We retarget the released Qwen3-4B DFlash and DeepSpec EAGLE-3 drafters to Qwen3-8B and Qwen3-14B (Yang et al., 2025; DeepSeek-AI, 2026). DFlash uses block length 16. EAGLE-3 uses the supported chain procedure with seven draft positions. Native controls use released target-specific drafters within the same implementation and proposal setting. The main AR comparisons use four NVIDIA L40S GPUs, with one sequential request per worker. Historical block-size and isolated-memory studies use RTX 6000 Ada GPUs. We compare methods within runs on the same hardware. Serving systems such as vLLM and SGLang also optimize cache use and request execution (Kwon et al., 2023; Zheng et al., 2024). Our timing isolates the stated per-request decoder rather than a concurrent serving workload.

Workloads and data use. We evaluate math, code and conversation workloads because speculative speed varies across tasks (Xia et al., 2024; Abramovich et al., 2026). MATH-500 supplies 500 questions (Lightman et al., 2024). The breadth suite contains 128 GSM8K questions, 164 HumanEval tasks, 378 MBPP tasks and 80 two-turn MT-Bench conversations (Hendrycks et al., 2021; Cobbe et al., 2021; Chen et al., 2021; Austin et al., 2021; Zheng et al., 2023). The 32-question development set informed interface and loss choices. Historical evaluations retain this exposure history. Fitting

Table 3: MATH-500 end-to-end tokens/s, including prompt processing. Progress R is mean recorded relay progress per cycle. Ratios and intervals use the AR arm of that same run. Native-drafter comparisons are in Table 5.

Drafter	Target	AR tok/s	Source tok/s	Relay tok/s	Relay / AR [95% CI]	Progress R
DFlash	8B	37.37	121.81	186.49	4.99 [4.88, 5.10]	6.98
DFlash	14B	22.88	92.15	116.88	5.11 [4.98, 5.23]	6.49
EAGLE-3	8B	36.15	66.44	84.92	2.35 [2.32, 2.38]	4.33
EAGLE-3	14B	22.88	53.96	59.18	2.59 [2.55, 2.63]	4.01

Table 4: Original drafter training and additional target adaptation. Record counts describe different training stages and objectives.

Stage	Records	Relative to relay fit	Weights learned
Published DFlash recipe (Chen et al., 2026)	∼800,000	∼195×	Drafter
Published EAGLE-3 recipe (Li et al., 2025)	∼532,000	∼130×	Drafter
RelaySpec target adaptation	4,096	1×	Feature map

and evaluation problem texts have zero normalized exact matches. Appendix E quantifies related templates using token overlap. All completed workload cells in each reported suite are included.

Quality and pairing. We score the same math outputs used for the AR timing comparison with the pinned Qwen math scorer. Historical code comparisons use EvalPlus base and expanded tests (Liu et al., 2023). MT-Bench measures two-turn generation time, with each method conditioning its second turn on its own first answer. Its bootstrap unit is the conversation. Other confidence intervals resample matched requests and condition on the fitted checkpoint. Model loading and external task scoring sit outside the request timer.

5.2 Acceleration over plain autoregressive decoding

Table 3 reports absolute end-to-end throughput and AR-relative speed. DFlash RelaySpec reaches 186.49 tokens/s at 8B and 116.88 at 14B, compared with AR at 37.37 and 22.88. EAGLE-3 RelaySpec reaches 84.92 and 59.18 tokens/s, compared with 36.15 and 22.88 for AR. The resulting throughput ratios are 4.99, 5.11, 2.35 and 2.59. Each paired interval is above one.

DFlash's parallel proposal step and longer accepted progress yield higher throughput than the sequential EAGLE-3 path in these implementations. This is a comparison of the stated released models and decoding settings. The two integrations use different pinned Transformers versions, so their cross-family difference is not an isolated architectural effect.

5.3 Retaining target-specific drafter performance

For the motivating 4B-to-8B case, RelaySpec reaches 186.49 tokens/s while native DFlash-8B reaches 206.58. Thus the reused 4B drafter retains **90.3%** of native throughput after fitting only the map. EAGLE-3 retains 90.2% at 8B and 89.1% at 14B. Figure 4 places this performance beside the scale of the additional fitting stage.

The DFlash recipe reports about 800K target-generated records. The EAGLE-3 recipe uses about 68K ShareGPT and 464K UltraChat entries. Relay fitting uses 0.51% and 0.77% of these respective record counts. This comparison describes marginal adaptation: the original drafter's training remains inherited. Record lengths, objectives and hardware differ, so these ratios do not represent equivalent reductions in training compute. The evaluated EAGLE-3 checkpoints come from DeepSpec, while the table's EAGLE-3 count refers specifically to the published training recipe.

5.4 Acceptance and the source-removal mechanism

Table 5 pairs retained throughput with accepted progress. RelaySpec advances 6.98 tokens per DFlash-8B cycle versus 7.89 for its native drafter. EAGLE-3 advances 4.33 versus 5.40 at 8B

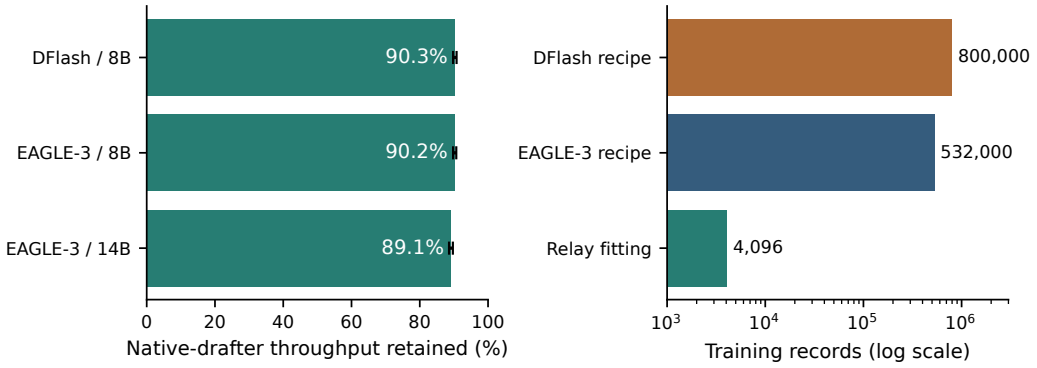

Figure 4: Left: fraction of native target-specific drafter throughput retained by RelaySpec in paired MATH-500 runs. Right: published original DFlash and EAGLE-3 recipe sizes versus the additional RelaySpec fitting set. Recipe counts provide training-scale context, not a matched training experiment or an audit of each released checkpoint.

Table 5: Native throughput retained and mean recorded progress per cycle. N and R denote native drafting and RelaySpec. Progress includes the implementation's target-supplied token.

Drafter	Target	Native tok/s	Relay tok/s	Retained [95% CI]	Progress N	Progress R
DFlash	8B	206.58	186.49	90.3% [89.8, 90.8]	7.89	6.98
EAGLE-3	8B	94.12	84.92	90.2% [89.8, 90.7]	5.40	4.33
EAGLE-3	14B	66.39	59.18	89.1% [88.6, 89.7]	5.28	4.01

and 4.01 versus 5.28 at 14B. Acceptance and speed retention therefore answer different questions: per-cycle work also changes with drafter size and implementation.

Against source reuse, the relay's throughput ratios are 1.531 and 1.268 for DFlash, and 1.278 and 1.097 for EAGLE-3, at 8B and 14B respectively. This secondary comparison holds the inherited drafter fixed and isolates source replacement. The source's computation disappears from each cycle, while the target's retained states already contain useful context. The matched EAGLE-3 normalization result in Section 5.8 supports the importance of supplying that context at its expected scale.

5.5 Math accuracy and output agreement

Table 6 scores the exact outputs timed in Table 3. Observed AR-to-relay accuracy differences range from -0.2 to $+0.8$ percentage points, and each paired interval includes zero. These measured outcomes support the accuracy comparison without treating token equality as a substitute for task quality.

In BF16, AR and block verification can select different tokens on an identical prefix. Traces of eight selected DFlash early-divergence prompts show changes in the leading target scores shared by native drafting, source reuse and RelaySpec. This diagnostic is separate from the timed runs. We report measured quality and agreement rather than claiming bitwise identity from the verifier's mathematical contract.

5.6 Speed across workloads

Figure 5 and Table 9 report the completed DFlash AR-paired breadth suite. RelaySpec is 1.85–3.69 times faster than AR across these eight cells. Code gains at 14B remain substantial relative to AR even where source reuse is faster than the relay. The useful comparison depends on the deployment choice: AR measures the acceleration obtained, while source reuse measures the benefit of removing its source.

Table 6: MATH-500 accuracy in percent. Difference is RelaySpec minus AR in percentage points. Token matches count identical full output sequences.

Drafter	Target	AR score	Relay score	Difference [95% CI]	Token matches
DFlash	8B	75.8	76.6	+0.8 [-1.4, +3.0]	117/500
DFlash	14B	79.2	79.2	+0.0 [-2.2, +2.0]	113/500
EAGLE-3	8B	75.8	75.6	-0.2 [-2.4, +2.0]	135/500
EAGLE-3	14B	79.2	79.4	+0.2 [-1.8, +2.2]	106/500

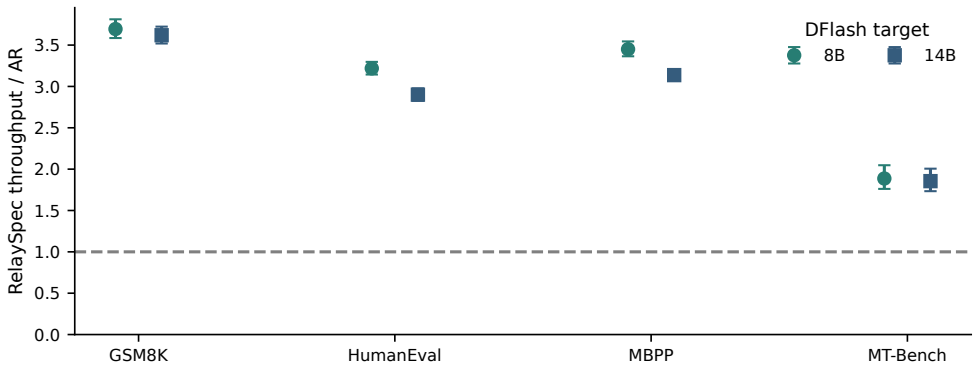

Figure 5: DFlash RelaySpec throughput relative to AR across workloads. Whiskers are paired 95% intervals. Chat intervals resample whole two-turn conversations. Full absolute values are in Table 9.

5.7 Fitting work and deployment memory

The selected maps have 52.43–65.54M trainable weights. Recorded fitting loops take 85.6–118.5 seconds on four RTX 6000 Ada GPUs with models already loaded. The interval covers feature computation and optimization. Loading, checkpoint writing and deployment initialization are separate setup costs. Appendix F gives each measured fitting loop.

Separate EAGLE-3 processes measure memory after removing source layers. Peak allocated memory falls from 25.56 to 17.76 GiB at 8B and from 37.56 to 29.94 GiB at 14B. These are savings of 7.80 and 7.63 GiB. Source token embeddings and vocabulary projections required by a drafter remain part of deployment. The claim concerns the removed source transformer, not every tensor inherited from the source.

5.8 Ablation studies

5.8.1 Fitting data and continued optimization

The completed DFlash-8B budget sweep uses 512, 1,024 and 2,048 distinct records for one pass, followed by a configuration with two passes over 4,096 records. The update counts are 128, 256, 512 and 2,048 with four records per update. Figure 6 shows throughput and progress on the same 128-question evaluation set. This measures a joint increase in data coverage and optimization work.

Throughput rises from 136.78 to 162.04, 173.72 and 187.85 tokens/s as the budget grows. Accepted progress rises with it. More fitting can improve the context supplied to a fixed drafter, reducing the number of cycles required for generation. The last configuration continues fitting over an already used manifest, so its gain reflects additional optimization as well as the larger distinct set. Table 12 reports all configurations and their actual budgets.

5.8.2 Draft length and interface choices

The block-size experiment directly compares AR, native DFlash and the relay at lengths 8, 16 and 32 on 64 fixed requests. Block 16 gives the highest relay throughput, 210.15 tokens/s. Block 8 retains

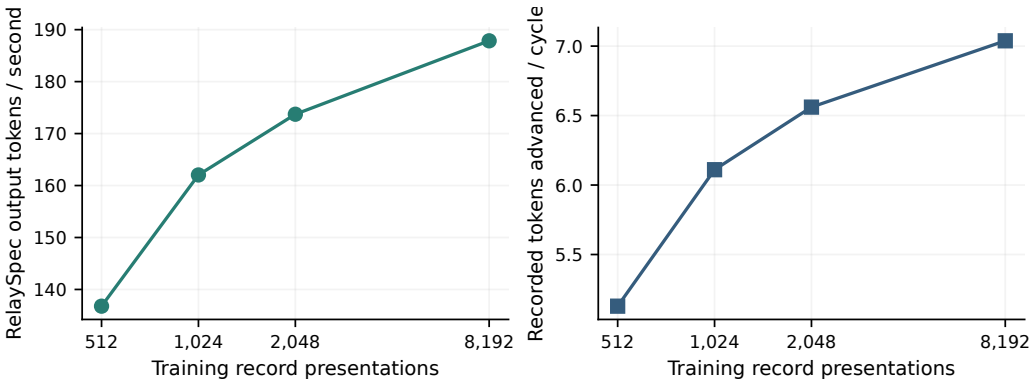

Figure 6: DFlash-8B fitting-budget sweep on 128 fixed questions. The first three points use one pass over 512, 1,024 and 2,048 distinct records. The last uses 8,192 presentations of 4,096 distinct records. Each point is a separately fitted checkpoint. Lines connect measured budgets.

Table 7: DFlash-8B block-size ablation on 64 questions, a 512-token cap and RTX 6000 Ada hardware. AR is measured in each run.

Block	AR tok/s	Native draft tok/s	Relay tok/s	Relay / AR	Progress / cycle
8	44.68	175.12	164.90	3.691	4.98
16	44.67	229.87	210.15	4.704	6.48
32	44.76	207.70	131.36	2.935	4.14

more source progress proportionally but advances fewer tokens per cycle. Block 32 adds verification width while reducing accepted progress for the released block-16 drafter. This supports keeping its trained block length instead of maximizing the relative gain over source reuse.

The EAGLE-3 normalization comparison holds parameter count, data and 128-update budget fixed. Raw target features reduce relative context error from 0.413 to 0.266 and increase the source-relative throughput ratio from 1.035 to 1.237. The raw interface consumes feature magnitude, which input normalization removes. This gives a concrete reason for preserving scale in the selected map.

For DFlash, the selected relative-error objective improves throughput by 1.009 and 1.011 times over squared error plus cosine distance on the 32-question comparison at 8B and 14B. The ridge and nonlinear variants in Appendix C provide additional fitting alternatives. Their different objectives and capacities make them comparisons of complete recipes rather than isolated tests of the optimizer or nonlinearity.

6 Conclusion

RelaySpec targets a specific cost: adapting an already trained feature-conditioned drafter to a new target. A fitted map retains the drafter and replaces its source-transformer conditioning. The measured Qwen3 comparisons achieve 2.35–5.11 times AR throughput and retain 89.1–90.3% of native target-specific drafter throughput using 4,096 additional fitting records. Data-budget and interface studies explain how accepted progress improves, while source removal reduces cycle cost and memory. Task quality and numerical agreement are reported alongside speed.

Scope. The evidence concerns fixed Qwen3 checkpoints, shared vocabularies and greedy single-request decoding. Confidence intervals condition on the fitted map. The fitting-budget sweep varies data and optimization jointly. These conditions define the comparisons supported by the reported measurements.

AI USE STATEMENT

Generative AI assisted with implementation, code review, literature search, analysis and manuscript editing. Numerical tables and plots are generated from recorded artifacts with checked method, request and scorer identity. Scientific interpretation and submission decisions remain the authors' responsibility.

ETHICS STATEMENT

This study evaluates inference methods using existing model checkpoints and benchmark datasets. It reports task quality alongside speed and retains the target's verification role. Faster generation retains the underlying models' potential biases and misuse risks. Use of the models and data remains subject to their respective licenses and terms.

REPRODUCIBILITY STATEMENT

The appendix specifies model identifiers, feature layers, fitting settings, measurement boundaries, complete workload results and data checks. The accompanying implementation includes configurations, request records, scorer provenance and scripts that regenerate the reported assets.

REFERENCES

Talor Abramovich, Maor Ashkenazi, Izzy Putterman, Benjamin Chislett, Tiyasa Mitra, Bita Darvish Rouhani, Ran Zilberstein, and Yonatan Geifman. SPEED-Bench: A unified and diverse benchmark for speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2604.09557. Accepted paper.

Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost PARallel draft model adaptation. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=XbOyv7iVGL.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL https://arxiv.org/abs/2108.07732.

Yamini Bansal, Preetum Nakkiran, and Boaz Barak. Revisiting model stitching to compare neural representations. In *Advances in Neural Information Processing Systems 34*, 2021. URL https://papers.nips.cc/paper/2021/hash/01ded4259d101feb739b06c399e9cd9c-Abstract.html.

Frédéric Berdoz, Peer Rheinboldt, and Roger Wattenhofer. Steering pretrained drafters during speculative decoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 30067–30075, 2026. doi: 10.1609/aaai.v40i36.40255. URL https://ojs.aaai.org/index.php/AAAI/article/view/40255.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://proceedings.mlr.press/v235/cai24b.html.

Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *43rd International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2602.06036. Accepted paper.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

DeepSeek-AI. DeepSpec: A full-stack codebase for training and evaluating speculative decoding draft models. Official open-source software, 2026. URL https://github.com/deepseek-ai/DeepSpec.

Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. URL https://aclanthology.org/2024.acl-long.681/.

Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://proceedings.mlr.press/v235/fu24a.html.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems 34, Datasets and Benchmarks Track*, 2021. URL https://openreview.net/forum?id=7Bywt2mQsCe.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, 2019. URL https://proceedings.mlr.press/v97/houlsby19a.html.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022. URL https://www.microsoft.com/en-us/research/publication/lora-low-rank-adaptation-of-large-language-models/.

Shijing Hu, Jingyang Li, Xingyu Xie, Zhihui Lu, Kim-Chuan Toh, and Pan Zhou. GRIFFIN: Effective token alignment for faster speculative decoding. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/b6e67ae290635d0874c4cb43ba2a2cfb-Abstract-Conference.html.

Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. RepSpec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554c1f-Abstract-Conference.html.

Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/1b96f01343ff10150e6719eb163e1536-Abstract-Conference.html.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. URL https://doi.org/10.1145/3600006.3613165.

Haodi Lei, Yafu Li, Haoran Zhang, Shunkai Zhang, Qianjia Cheng, Xiaoye Qu, Ganqu Cui, Bowen Zhou, Ning Ding, Yun Luo, and Yu Cheng. Draft-OPD: On-policy distillation for speculative draft models. *arXiv preprint arXiv:2605.29343*, 2026. URL https://arxiv.org/abs/2605.29343.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 19274–19286, 2023. URL https://proceedings.mlr.press/v202/leviathan23a.html.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, 2024a. URL https://proceedings.mlr.press/v235/li24bt.html.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024b. URL https://aclanthology.org/2024.emnlp-main.422/.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems 38*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/c7b5a35ea98b62512a869c19ea7b03cb-Abstract-Conference.html.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/aca97732e30bcf1303bc22ac3924fd16-Abstract-Conference.html.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems 36*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024. URL https://arxiv.org/abs/2305.09781.

Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. OmniDraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3c2fe1417eed1c6ff9acf169617981ea-Abstract-Conference.html.

Damian Smith, Harvey Mannering, and Antonia Marcu. Functional alignment can mislead: Examining model stitching. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 55972–55998, 2025. URL https://proceedings.mlr.press/v267/smith25a.html.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise Parallel Decoding for Deep Autoregressive Models. In *Advances in Neural Information Processing Systems 31*, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/c4127b9194fe8562c64dc0f5bf2c93bc-Abstract.html.

Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Gaurav Jain, Oren Pereg, Moshe Wasserblat, and David Harel. Accelerating LLM inference with lossless speculative decoding algorithms for heterogeneous vocabularies. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025a. URL https://proceedings.mlr.press/v267/timor25a.html.

Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Oren Pereg, Moshe Wasserblat, Tomer Galanti, Michal Gordon-Kiwkowitz, and David Harel. Distributed speculative inference: Speculation parallelism for provably faster lossless language model inference. In *International Conference on Learning Representations*, 2025b. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/b36554b97da741b1c48c9de05c73993e-Abstract-Conference.html.

Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. OPT-Tree: Speculative Decoding with Adaptive Draft Tree Structure. *Transactions of the Association for Computational Linguistics*, 2025. URL https://aclanthology.org/2025.tacl-1.8/.

Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu Wei, and Zhifang Sui. Speculative Decoding: Exploiting Speculative Execution for Accelerating Seq2seq Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. URL https://aclanthology.org/2023.findings-emnlp.257/.

Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. URL https://aclanthology.org/2024.findings-acl.456/.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL https://arxiv.org/abs/2505.09388.

Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Advances in Neural Information Processing Systems 32*, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & Verify: Lossless Large Language Model Acceleration via Self-Speculative Decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. URL https://aclanthology.org/2024.acl-long.607/.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 37*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

Xiandong Zou, Jianshu Li, Jing Huang, and Pan Zhou. Variational speculative decoding: Rethinking draft training from token likelihood to sequence acceptance. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL https://ink.library.smu.edu.sg/sis_research/11190/.

APPENDIX

The appendix gives implementation details (Appendix A), complete workload results (Appendix B), fitting studies (Appendix C), acceptance measurements (Appendix D), data checks (Appendix E) and fitting-time and memory measurements (Appendix F).

A IMPLEMENTATION AND MEASUREMENT DETAILS

Released models. The source is `Qwen/Qwen3-4B`. Targets are `Qwen/Qwen3-8B` and `Qwen/Qwen3-14B`. The source DFlash is `z-lab/Qwen3-4B-DFlash-b16`, and its native 8B control is `z-lab/Qwen3-8B-DFlash-b16`. EAGLE-3 uses DeepSpec's `eagle3_qwen3_4b_ttt7`, `eagle3_qwen3_8b_ttt7` and `eagle3_qwen3_14b_ttt7` releases. All model and upstream revisions are fixed in the accompanying configurations.

Features and dimensions. Layer indices count transformer blocks from zero. Source and Qwen3-8B states come from blocks $[1, 9, 17, 25, 33]$. Qwen3-14B states come from $[1, 10, 19, 28, 37]$. Joining five 4,096-wide states gives a 20,480-wide 8B input. Joining five 5,120-wide states gives a 25,600-wide 14B input. Both drafter interfaces have width 2,560. The matrix dimensions are therefore $2,560 \times 20,480$ and $2,560 \times 25,600$. The DFlash input normalization has no trainable gain or bias. The released output normalization remains frozen.

Training. Four workers average record losses at each update. Seed 1729 fixes the selected fit. AdamW uses learning rate 0.0006, zero weight decay and gradient clipping at 1.0. Computation uses BF16, with the feature loss computed in FP32. Each of 1,024 updates processes four records, each capped at 192 rendered tokens. The same map remains fixed across tasks.

Runtime and timing. The main AR runs use Python 3.12.13 and PyTorch 2.9.1 with CUDA 12.8. DFlash uses Transformers 5.3.0. EAGLE-3 uses the pinned 5.10.2 overlay. Each worker owns a full target replica on one L40S GPU. Four workers process independent requests rather than splitting one model over GPUs. One warmup per method precedes measured requests. Method order rotates across requests. The synchronized timer includes prompt processing, drafting, verification, feature updates and runtime bookkeeping. The historical block and memory experiments use RTX 6000 Ada GPUs.

Stopping and conversation history. Generation stops at the first end token or the output cap. Capped requests remain in both timing and quality aggregates. Each MT-Bench method uses its own first-turn output as second-turn context. Two turns form one resampling unit. The breadth run has 750 initial records and 830 timed requests after expanding 80 conversations to two turns.

Aggregation. Each method's throughput is the sum of its output-token counts divided by the sum of its request times. Each of 10,000 resamples recomputes this ratio from matched requests, using bootstrap seed 1729. Confidence bounds are the central 95% of resampled estimates. The request-time ratio uses summed times alone. Table 8 reports both measurements and output lengths.

Table 8: Additional MATH-500 measurements. A is AR, S is source reuse and R is RelaySpec. Time ratio divides summed AR time by summed relay time.

Drafter	Target	Mean tokens AR / R	Time ratio AR / R	Throughput R / S	Progress S / R
DFlash	8B	783.22 / 778.81	5.018	1.531	7.66 / 6.98
DFlash	14B	777.66 / 772.33	5.143	1.268	7.44 / 6.49
EAGLE-3	8B	783.22 / 767.79	2.397	1.278	5.15 / 4.33
EAGLE-3	14B	777.66 / 766.20	2.625	1.097	5.07 / 4.01

B COMPLETE WORKLOAD RESULTS

The earlier fixed-map source comparisons in Table 10 provide all four workloads for both drafter families on RTX 6000 Ada. They are a separate paired experiment with their own denominator.

Table 9: Complete DFlash breadth comparisons with paired AR. Ratios use end-to-end throughput. Every planned task and target cell is included.

Target	Task	Requests	AR tok/s	Relay tok/s	Relay / AR [95% CI]	Relay / source
8B	GSM8K	128	37.60	138.89	3.69 [3.59, 3.81]	1.456
8B	HumanEval	164	37.55	120.87	3.22 [3.14, 3.30]	1.244
8B	MBPP	378	37.49	129.35	3.45 [3.36, 3.54]	1.320
8B	MT-Bench	160	37.47	70.69	1.89 [1.76, 2.05]	1.387
14B	GSM8K	128	26.80	96.98	3.62 [3.52, 3.72]	1.104
14B	HumanEval	164	26.73	77.54	2.90 [2.83, 2.98]	0.890
14B	MBPP	378	26.72	83.82	3.14 [3.07, 3.21]	0.956
14B	MT-Bench	160	26.55	49.26	1.85 [1.73, 2.01]	1.086

The EAGLE-3 14B code cells show the reduced margin when lower accepted progress outweighs source-removal savings. This operating range complements the AR acceleration measured in the main comparison.

Table 10: Historical paired source-reuse breadth measurements on RTX 6000 Ada. Ratios compare relay with source throughput within these runs. They are not normalized by an AR arm from another experiment.

Drafter	Target	Task	Requests	Source tok/s	Relay tok/s	Ratio [95% interval]
DFlash	8B	GSM8K	128	116.87	166.39	1.4237 [1.4040, 1.4445]
DFlash	8B	HumanEval	164	118.84	145.00	1.2201 [1.2017, 1.2383]
DFlash	8B	MBPP	378	117.77	151.55	1.2868 [1.2732, 1.3001]
DFlash	8B	MT-Bench	160	60.85	81.62	1.3412 [1.3183, 1.3639]
DFlash	14B	GSM8K	128	88.04	97.07	1.1025 [1.0836, 1.1214]
DFlash	14B	HumanEval	164	87.34	77.61	0.8886 [0.8741, 0.9038]
DFlash	14B	MBPP	378	88.02	83.98	0.9541 [0.9416, 0.9663]
DFlash	14B	MT-Bench	160	45.32	49.30	1.0877 [1.0653, 1.1083]
EAGLE-3	8B	GSM8K	128	85.99	108.45	1.2613 [1.2437, 1.2789]
EAGLE-3	8B	HumanEval	164	71.75	81.10	1.1304 [1.1172, 1.1434]
EAGLE-3	8B	MBPP	378	69.85	82.10	1.1753 [1.1656, 1.1850]
EAGLE-3	8B	MT-Bench	160	42.27	49.45	1.1698 [1.1501, 1.1906]
EAGLE-3	14B	GSM8K	128	67.81	73.41	1.0826 [1.0675, 1.0979]
EAGLE-3	14B	HumanEval	164	55.38	50.58	0.9132 [0.9011, 0.9255]
EAGLE-3	14B	MBPP	378	54.55	50.49	0.9256 [0.9151, 0.9361]
EAGLE-3	14B	MT-Bench	160	33.83	34.57	1.0221 [0.9999, 1.0449]

Table 11: Task scores for the historical fixed-map breadth runs. Source reuse and RelaySpec share each reported score. Code columns are single-program pass percentages under base and expanded EvalPlus tests.

Drafter	Target	GSM8K	HumanEval	HumanEval+	MBPP	MBPP+
DFlash	8B	93.75	85.98	80.49	84.13	73.02
DFlash	14B	94.53	89.02	83.54	88.36	75.13
EAGLE-3	8B	92.97	87.20	82.32	83.07	71.96
EAGLE-3	14B	94.53	90.24	85.98	88.89	74.87

Across these historical math/code comparisons, all 4,680 paired task outcomes agree between source reuse and RelaySpec. This count is four model/drafter pairs times $500 + 128 + 164 + 378 = 1,170$ problems. Expanded code tests evaluate the same programs and do not add independent examples. For the current AR-paired DFlash GSM8K runs, the scorer gives 120/128 correct for all arms at 8B and 121/128 at 14B. Code scores above refer specifically to the historical outputs, preserving scorer/run identity.

Table 12: Completed DFlash-8B fitting studies on the same 128-question set. Records count distinct fitting examples. Two passes over 4,096 records give 8,192 presentations. Source reuse is paired within each run.

Fitting choice	Records	Updates	Relay tok/s	Relay / source [95% CI]	Score
512 records	512	128	136.78	1.150 [1.118, 1.181]	82.03
1,024 records	1,024	256	162.04	1.372 [1.347, 1.397]	82.03
2,048 records	2,048	512	173.72	1.477 [1.458, 1.495]	82.03
4,096 records, two passes	4,096	2,048	187.85	1.573 [1.558, 1.588]	82.03
Ridge surrogate	4,096	*solve*	115.18	0.955 [0.927, 0.982]	82.03
Nonlinear width 512	4,096	1,024	104.49	0.872 [0.841, 0.903]	82.03

C FITTING AND INTERFACE STUDIES

The data-budget configurations use the same normalized linear interface and relative-error objective. The nonlinear map uses an intermediate width of 512 and about 11.8M parameters, compared with 52.4M for the dense 8B map. Its lower throughput is consistent with a less effective fitted interface, but the comparison also changes capacity. It does not isolate a disadvantage of nonlinear functions. The ridge configuration solves a raw linear surrogate with ridge coefficient 1.0. Since the deployed DFlash interface includes output normalization, this is a different objective from Equation 4.

Table 13: Matched EAGLE-3 8B normalization comparison on 32 development questions after 128 updates. Throughput ratios use source reuse.

Input to linear map	Relative error	Throughput ratio [95% interval]	Progress retained
Normalized features	0.413	1.035 [1.003, 1.068]	69.0%
Raw features	0.266	1.237 [1.213, 1.263]	82.6%

Table 14: DFlash objective comparison on 32 development questions. Candidates share fitting data, update count, normalization and optimizer. Each ratio uses the source-reuse arm of that candidate's paired run.

Target	Objective	Throughput ratio [95% interval]	Token matches
8B	Relative error	1.523 [1.488, 1.558]	32/32
8B	Squared error + cosine	1.509 [1.476, 1.545]	32/32
14B	Relative error	1.265 [1.243, 1.289]	32/32
14B	Squared error + cosine	1.251 [1.229, 1.276]	32/32

Relative error weights a record's feature mismatch by teacher magnitude. Squared error plus 0.1 times cosine distance combines coordinate error with directional agreement. Direct candidate-to-candidate throughput ratios are 1.0085 [1.0005, 1.0174] at 8B and 1.0114 [1.0046, 1.0177] at 14B. These are modest gains with one fewer mixing coefficient.

D ACCEPTANCE AND NUMERICAL AGREEMENT

We pool all recorded proposal cycles rather than averaging each request's mean. This weights cycles equally. DFlash's recorded length is one already target-selected position plus the accepted draft prefix. EAGLE-3's length follows its shared decoder's progress count. These quantities measure work advanced per cycle, including target-supplied progress. A proposed-token acceptance fraction requires separating those positions and accounting for the number proposed.

The main MATH-500 table reports measured full-sequence agreement with AR. A fresh diagnostic selects eight prompts with early differences and saves target inputs, cache lengths, positions and the two leading logits at every call. The DFlash paths share the same first differing choice across native, source and relay drafting. The observed BF16 logits include ties or changed ordering with gaps up to 0.25 at those positions. Such traces identify numerical score differences in the selected cases. They

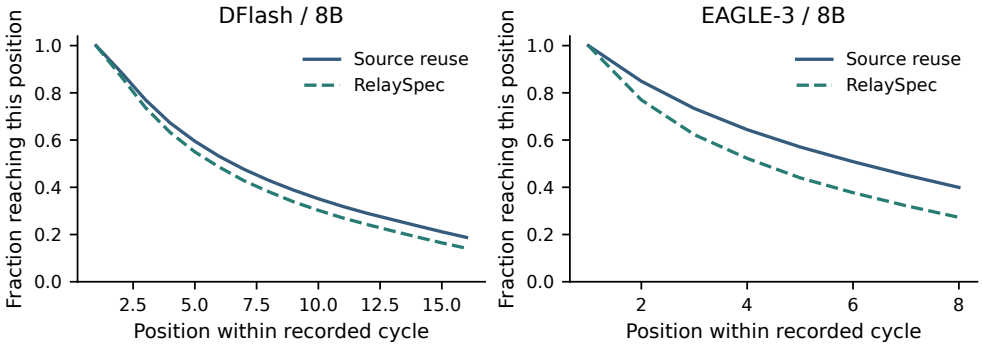

Figure 7: Historical 8B source-reuse comparisons. Each curve gives the fraction of recorded cycles reaching each position. Summing the curve values gives mean recorded progress per cycle. Solid and dashed lines show source reuse and RelaySpec, respectively.

do not establish that all mismatches have the same cause. The task-quality intervals in Table 6 remain the empirical quality evidence.

E Data checks and development exposure

The 4,096-record fitting manifest contains distinct problems with solutions. The loader renders both fields before applying the 192-token cap. Repeating the manifest changes presentations, not distinct records. The 32-question development set informed normalization and objective choices. The historical 468-question complement is computed from saved full-run outputs and retains that exposure history.

Normalized exact matching finds zero overlaps between the 4,096 fitting and 1,250 evaluation problem texts. For a second check, each text is split into unique normalized tokens. Token-set Jaccard overlap is the number of shared tokens divided by the number of tokens in either set. Each evaluation question is compared with its closest fitting record. Table 15 reports the resulting similarities.

Table 15: Closest fitting-to-evaluation token-set overlap. Threshold columns count evaluation records at or above the stated overlap.

Task	Maximum overlap	≥ 0.80	≥ 0.90	≥ 0.95
MATH-500	0.980	47	13	4
GSM8K	0.378	0	0	0
HumanEval	0.290	0	0	0
MBPP	0.333	0	0	0
MT-Bench	0.571	0	0	0

MATH-500 includes 47 questions above 0.80 overlap and four above 0.95. These shared templates qualify the interpretation of math generalization. The fixed-map code and conversation evaluations give additional task coverage without changing the fitted map.

F FITTING TIME AND ISOLATED MEMORY

Table 16: Selected map size and warm fitting-loop duration on four RTX 6000 Ada GPUs. Models are loaded before the recorded loop begins.

Drafter	Target	Trainable weights (millions)	Fitting loop (seconds)
DFlash	8B	52.43	85.6
DFlash	14B	65.54	118.0
EAGLE-3	8B	52.43	86.7
EAGLE-3	14B	65.54	118.5

The fitting-loop timer covers source and target feature computation and matrix optimization. A complete setup also loads model weights, writes the map, reloads it and initializes generation. Published original training counts in Table 4 provide context for the additional-data requirement. They are not a wall-time or token-budget comparison with fitting a new native drafter.

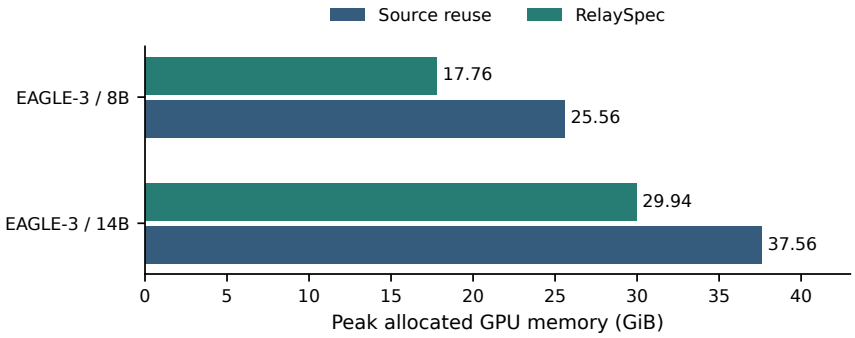

Figure 8: Isolated EAGLE-3 peak allocated memory on RTX 6000 Ada. Each method runs in a fresh process on eight paired prompts per target. Source removal saves 7.80 GiB at 8B and 7.63 GiB at 14B.

GPU peak counters are reset before the measured request. Allocated memory counts live tensor allocations and differs from reserved allocator memory. The mixed-arm processes used for timing are not the basis of this source-removal measurement.